beast网络库实现websocket服务器

Posted on 2023-06-27 09:51:03 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2Rlhdbut49eOVgjlVq9aj6nF7Rg)

简介

使用beast网络库实现websocket服务器，一般来说websocket是一个长连接的协议，但是自动包含了解包处理，当我们在浏览器输入一个http请求时如果是以ws开头的
如 `ws://127.0.0.1:9501` 就是请求本地9501端口的websocket服务器处理。而beast为我们提供了websocket的处理方案，我们可以在http服务器的基础上升级协议为websocket，处理部分websocket请求。如果服务器收到的是普通的http请求则按照http请求处理。我们可以从官方文档中按照示例逐步搭建websocket服务器。

构造websocket

在开始前我们先准备几个变量

```
#include <boost/beast.hpp>
#include <boost/beast/ssl.hpp>
#include <boost/asio.hpp>
#include <boost/asio/ssl.hpp>
namespace net = boost::asio;
namespace beast = boost::beast;
using namespace boost::beast;
using namespace boost::beast::websocket;

net::io_context ioc;
tcp_stream sock(ioc);
net::ssl::context ctx(net::ssl::context::tlsv12);
```

WebSocket连接需要一个有状态对象，由Beast中的一个类模板websocket::stream表示。该接口使用分层流模型。一个websocket stream对象包含另一个流对象，称为“下一层”，它用于执行I/O操作。以下是每个模板参数的描述：

```
namespace boost {
namespace beast {
namespace websocket {

template<
    class NextLayer,
    bool deflateSupported = true>
class stream;

} // websocket
} // beast
} // boost
```

这段代码定义了Beast库中WebSocket实现的命名空间。其中， `websocket` 命名空间下包含一个模板类 `stream` ，用于表示WebSocket连接。

`stream` 类有两个模板参数： `NextLayer` 和 `deflateSupported` 。其中， `NextLayer` 表示WebSocket连接使用的下一层流类型，例如TCP套接字或TLS握手后的数据流；
而 `deflateSupported` 则是一个bool值，表示是否支持WebSocket协议内置的压缩功能。

这些代码所在的三个命名空间分别是 `boost` 、 `beast` 和 `websocket` ，是为了防止与其他库或用户代码发生名称冲突而创建的。将Beast库放在 `beast` 命名空间下是为了与Boost库本身分离，方便管理和维护。
当创建一个WebSocket流对象时，构造函数提供的任何参数都会被传递给下一层对象的构造函数。以下示例代码声明了一个基于TCP/IP套接字和I/O上下文的WebSocket流对象：

```
stream<tcp_stream> ws(ioc);
```

上述代码创建了一个基于TCP流的WebSocket流对象，使用了指定的I/O上下文，该代码中的stream是Beast库中WebSocket流类模板的别名，其下一层流类型为tcp_stream。

需要注意的是，WebSocket流使用自己特定的超时功能来管理连接。如果使用tcp_stream或basic_stream类模板与WebSocket流一起使用，那么在连接建立后应该禁用TCP或basic流上的超时设置，否则流的行为将是不确定的。

这是因为WebSocket协议本身包含了超时机制，当流上发生超时时，WebSocket库会优先处理超时并关闭连接，而不会将超时事件传递给下层TCP或basic流。如果同时在WebSocket和TCP或basic流上启用超时设置，就可能出现冲突和未定义的行为。

因此，当使用WebSocket流时，应该避免在底层的TCP或basic流上设置超时，而是可以通过

WebSocket流对象的 `set_option` 函数来设置WebSocket连接的超时时间。这样可以确保WebSocket连接中的超时机制正常工作，并且不会干扰底层流的超时设置。

与大多数I/O对象一样，WebSocket流对象也不是线程安全的。如果两个不同的线程同时访问该对象，则会产生未定义行为。

对于多线程程序，可以通过在构造tcp_stream对象时使用executor（如strand）来保证线程安全。下面的代码声明了一个使用strand来调用所有完成处理程序的WebSocket流：

```
1. stream<tcp_stream> ws(net::make_strand(ioc));
```

如果下一层流支持移动构造，那么WebSocket流对象可以从一个已移动的对象中构造。

这意味着，在创建WebSocket流对象时，可以将下一层流对象的所有权转移到WebSocket流对象中，而不需要进行复制或重新分配。这种方式可以避免额外的内存开销和数据拷贝操作，提高程序运行效率。

例如，可以使用std::move函数将一个已存在的TCP套接字对象移动到WebSocket流中：

```
1. stream<tcp_stream> ws(std::move(sock));
```

可以通过调用WebSocket流对象的next_layer函数来访问下一层流对象。

```
1. ws.next_layer().close();
```

使用SSL

使用 `net::ssl::stream` 类模板作为流的模板类型，并且
将 `net::io_context` 和 `net::ssl::context` 参数传递给包装流的构造函数。

```
1. stream<ssl_stream<tcp_stream>> wss(net::make_strand(ioc), ctx);
```

当然如果websocket stream 使用SSL类型需要包含 `<boost/beast/websocket/ssl.hpp>`
`next_layer()` 函数用于访问底层的 SSL 流。 `ssl::stream` 类中的 `next_layer()` 函数返回对底层 `ssl_stream` 的引用，它代表了建立在 `SSL/TLS` 层之上的网络流。

```
1. wss.next_layer().handshake(net::ssl::stream_base::client);
```

在上述声明的多层流（如 SSL 流）中，使用 next_layer 进行链式调用访问每个层可能会很麻烦。为了简化这个过程，Boost.Beast 提供了 get_lowest_layer() 函数，用于获取多层流中的最底层流。

通过调用 get_lowest_layer() 函数，您可以直接获取多层流中的最底层流，而无需逐层调用 next_layer()。这对于取消所有未完成的 I/O 操作非常有用，例如在关闭连接之前取消所有挂起的异步操作。

```
1. get_lowest_layer(wss).cancel();
```

连接

在进行 WebSocket 通信之前，需要首先连接 WebSocket 流，然后执行 WebSocket 握手。WebSocket 流将建立连接的任务委托给下一层流。例如，如果下一层是可连接的流或套接字对象，则可以访问它以调用必要的连接函数。以下是作为客户端执行的示例代码

```
1. stream<tcp_stream> ws(ioc);
2. net::ip::tcp::resolver resolver(ioc);
3. get_lowest_layer(ws).connect(resolver.resolve("example.com", "ws"));
```

对于服务器接收连接，在WebSocket服务器中使用一个acceptor来接受传入的连接。当建立了一个传入连接时，可以从acceptor返回的socket构造WebSocket流。

```
1. net::ip::tcp::acceptor acceptor(ioc);
2. acceptor.bind(net::ip::tcp::endpoint(net::ip::tcp::v4(), 0));
3. acceptor.listen();
4. stream<tcp_stream> ws(acceptor.accept());
```

也可以通过使用acceptor成员函数的另一个重载，将传入连接直接接受到WebSocket流拥有的socket中

```
1. stream<tcp_stream> ws(net::make_strand(ioc));
2. acceptor.accept(get_lowest_layer(ws).socket());
```

握手

websocket通过握手将http升级为websocket协议，一个websocket协议如下

```
GET / HTTP/1.1
Host: www.example.com
Upgrade: websocket
Connection: upgrade
Sec-WebSocket-Key: 2pGeTR0DsE4dfZs2pH+8MA==
Sec-WebSocket-Version: 13
User-Agent: Boost.Beast/216
```

先说客户端如何升级

这段代码使用websocket::stream的成员函数handshake和async_handshake，用于使用所需的主机名和目标字符串发送请求。该代码连接到从主机名查找返回的IP地址，然后在客户端角色中执行WebSocket握手

```
stream<tcp_stream> ws(ioc);
net::ip::tcp::resolver resolver(ioc);
get_lowest_layer(ws).connect(resolver.resolve("www.example.com", "ws"));
ws.handshake(
    "www.example.com",  // The Host field
    "/"                 // The request-target
);
```

在客户端收到来自服务器的HTTP Upgrade响应并指示成功升级时，调用者可能希望对接收到的HTTP响应消息进行额外的验证。例如，检查对基本身份验证挑战的响应是否有效。为了实现这一目的，handshake成员函数提供了重载形式，允许调用者将接收到的HTTP消息存储在类型为response_type的输出引用参数中。

```
response_type res;
ws.handshake(
    res,                // Receives the HTTP response
    "www.example.com",  // The Host field
    "/"                 // The request-target
);
```

所以上述握手函数根据自己的需求调用一个即可。

再说服务器如何升级

对于接受传入连接的服务器，websocket::stream可以读取传入的升级请求并自动回复。如果握手符合要求，流将发送带有101切换协议状态码的升级响应。如果握手不符合要求，或者超出了调用者之前设置的流选项允许的参数范围，流将发送一个带有表示错误的状态码的

HTTP响应。根据保持活动设置，连接可能保持打开状态，以进行后续握手尝试。在接收到升级请求握手后，由实现创建和发送的典型HTTP升级响应如下所示：

```
1. HTTP/1.1 101 Switching Protocols
2. Upgrade: websocket
3. Connection: upgrade
4. Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
5. Server: Boost.Beast
```

stream的accept和async_accept成员函数用于从已连接到传入对等方的流中读取WebSocket HTTP升级请求握手，然后发送WebSocket HTTP升级响应。示例如下

```
1. ws.accept();
```

在某些情况下，服务器可能需要从流中读取数据，并在稍后决定将缓冲的字节解释为WebSocket升级请求。为了满足这种需求，accept和async_accept提供了接受额外缓冲区参数的重载版本。

以下是一个示例，展示了服务器如何将初始的HTTP请求头部读取到一个动态缓冲区中，然后稍后使用缓冲的数据尝试进行WebSocket升级：

```
1. std::string s;
2. net::read_until(sock, net::dynamic_buffer(s), "\r\n\r\n");
3. ws.accept(net::buffer(s));
```

在实现同时支持WebSocket的HTTP服务器时，服务器通常需要从客户端读取HTTP请求。为了检测传入的HTTP请求是否是WebSocket升级请求，可以使用函数is_upgrade。

一旦调用者确定HTTP请求是WebSocket升级请求，就会提供额外的accept和async_accept重载版本，这些版本接收整个HTTP请求头作为一个对象，以进行握手处理。通过手动读取请求，程序可以处理普通的HTTP请求以及升级请求。程序还可以根据HTTP字段强制执行策略，例如基本身份验证。在这个示例中，首先使用HTTP算法读取请求，然后将其传递给新构建的流：

```
flat_buffer buffer;
http::request<http::string_body> req;
http::read(sock, buffer, req);
if(websocket::is_upgrade(req))
{
    stream<tcp_stream> ws(std::move(sock));

    BOOST_ASSERT(buffer.size() == 0);

    ws.accept(req);
}
else
{

}
```

所以综上所述，在构建websocket升级时，可以用两种方式，一种是websocket来accept，另一种是在处理http的请求时将请求升级为websocket。
而这两种我们在实战的代码中都有实现，可以下载源码查看。

收发数据

当我们建立好websocket的握手后，就可以通过读写操作收发数据了。

```
flat_buffer buffer;
ws.read(buffer);
ws.text(ws.got_text());
ws.write(buffer.data());
buffer.consume(buffer.size());
```

上面的代码片段采用同步读和同步写的方式，根据接收消息的类型设置 WebSocket 连接的模式。
回显接收到的消息给对等端。
清空缓冲区，以便下一次读取可以开始构建新的消息。

但有些场景我们不能通过buffer一次性的读出数据

这是一些使用场景，这些场景中无法预先缓冲整个消息：

1. 向终端流式传输多媒体：在流式传输多媒体到终端时，通常不适合或不可能预先缓冲整

个消息。例如，在实时视频流或音频流的传输过程中，数据可能以非常大的速率产生，并且需要立即传输给接收端进行实时播放。由于数据量巨大且需即时传输，预先缓冲整个消息可能会导致延迟或资源耗尽。

2. 发送超出内存容量的消息：有时候需要发送的消息太大，无法一次性完全存储在内存中。这可能发生在需要传输大型文件或大量数据的情况下。如果尝试将整个消息加载到内存中，可能会导致内存溢出或系统性能下降。在这种情况下，需要通过分块或逐步读取的方式来发送消息，以便逐步加载和传输数据。

3. 提供增量结果：在某些情况下，需要在处理过程中提供增量的结果，而不是等待整个处理完成后再返回结果。这可以在长时间运行的计算、搜索或处理任务中发生。通过逐步提供部分结果，可以让用户或应用程序更早地获得一些数据，并可以在处理过程中进行进一步的操作或显示。这种方式可以改善用户体验，并减少等待时间。

在这些特定的使用场景中，需要采用逐步处理、流式传输或增量输出的方式，而不是依赖于预先缓冲整个消息。这样可以提高性能、降低内存消耗，并满足特定需求。

如下是asio提供的官方案例，通过流式读取的方式获取对端信息

```
1.
2. multi_buffer buffer;
3.
4. do
5. {
6.     ws.read_some(buffer, 512);
7. }
8. while(! ws.is_message_done());
9.
10.
11. ws.binary(ws.got_binary());
12.
13.
14. buffers_suffix<multi_buffer::const_buffers_type> cb{buffer.data()};
15.
16. for(;;)
17. {
18.     if(buffer_bytes(cb) > 512)
19.     {
20.         ws.write_some(false, buffers_prefix(512, cb));
21.         cb.consume(512);
22.     }
23.     else
24.     {
25.         ws.write_some(true, cb);
26.         break;
27.     }
28. }
29.
30. buffer.consume(buffer.size());
```

这段代码涉及 WebSocket 数据的读取和写入操作。以下是对每个部分的解释：

1 `multi_buffer buffer;`
这行代码定义了一个名为 `buffer` 的 `multi_buffer` 对象，用于存储读取的 WebSocket 数据。

2 `do { ... } while(! ws.is_message_done());`
这部分代码使用一个循环，连续调用 ws.read_some 方法从 WebSocket 连接中读取数据，并将其存储到 buffer 中，直到 WebSocket 消息全部接收完成。

3 `ws.binary(ws.got_binary());`
此行代码将 WebSocket 连接设置为二进制模式，以便在后续的写入操作中正确处理二进制数据。

4 `buffers_suffix<multi_buffer::const_buffers_type> cb{buffer.data()};`
这行代码创建了一个 cb 对象，它是 buffer 的后缀子序列。它提供了对 buffer 中已接收数据的访问。

5 `if(buffer_bytes(cb) > 512) { ... } else { ... }`
这段代码检查 cb 中的数据量是否大于 512 字节。如果是，将执行 if 语句块；否则，将执行 else 语句块。

6 `ws.write_some(false, buffers_prefix(512, cb));`
如果 cb 中的数据量大于 512 字节，此行代码将发送 cb 的前缀（前 512 字节）到 WebSocket 连接中，并保留剩余的数据。

7 `cb.consume(512);`
这行代码告知 cb 对象消耗了前 512 字节的数据，以便在后续迭代中更新迭代范围。

8 `ws.write_some(true, cb);`
如果 cb 中的数据量少于等于 512 字节，此行代码将发送 cb 中的所有数据到 WebSocket 连接中。

buffer.consume(buffer.size());
这行代码清空 buffer 中存储的数据，使其为空。

总体来说，这段代码的作用是读取 WebSocket 数据并将其写回 WebSocket 连接，确保数据按照预期进行处理。最后，通过 buffer.consume(buffer.size()) 清空缓冲区，准备下一次数据读取操作。

关闭连接

当我们想关闭连接时，可以通过close 或 async_close 的关闭函数。
具体而言，WebSocket 协议规定了两种关闭方式：

`close`
这是一个同步操作的函数，用于关闭 WebSocket 会话。当调用 close 时，客户端或服务器会向对方发送一个关闭帧，并且在收到对方的关闭帧后，会话将被正常关闭。

async_close

这是一个异步操作的函数，用于关闭 WebSocket 会话。与 close 不同，async_close 是一个非阻塞操作，它不会等待对方的关闭帧，而是立即返回，并触发一个异步关闭操作。这样可以在关闭过程中继续处理其他任务，而不必等待关闭完成。

这些关闭函数允许主机在 WebSocket 会话中发起关闭请求，以便安全地终止连接。应用程序可以根据需要选择适合的关闭函数，具体取决于其对同步或异步关闭的要求。

因为一个websocket会包含多个层，所以我们可以通过获取最底层，再执行关闭。这样保证所有层都安全关闭。

```
1. get_lowest_layer(wss).close();
```

总结

上面讲述了beast网络库websocket的使用方法，我通过实战，利用异步函数封装了一个websocket服务器，可以从下面的源码连接里看看项目代码。

视频连接https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101)

源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(2824) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) 喜欢(583) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ^ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)